

# Day 5, Session 2: JIT GPU Acceleration (Numba) & OpenCL (PyOpenCL)

---

**Duration:** 2 hours (theory + live demonstrations)

**Prerequisites:** Session 1 (GPU architecture, thread/block hierarchy)

**Demos folder:** [day5/demo/](#)

---

## Table of Contents

1. JIT Compilation with Numba CUDA
    - Why Numba for GPUs?
    - The `@cuda.jit` Decorator
    - Thread Indexing with `cuda.grid()`
    - Shared Memory Optimization & Thread Synchronization
    - GPU-Accelerated Math Libraries
  2. Cross-Platform Heterogeneous Computing with PyOpenCL
    - Why OpenCL? The Vendor Independence Advantage
    - The OpenCL Hardware & Platform Model
    - Compiling and Running OpenCL Kernels
    - Evaluating Expressions and Testing OpenCL Applications
  3. CUDA vs OpenCL vs CPU Concurrency: Final Comparison
  4. 5-Day Course Summary & Grand Wrap-Up
- 

## 1. JIT Compilation with Numba CUDA

### Why Numba for GPUs?

In PyCUDA, you write your kernel as a **C string** inside Python:

```
# PyCUDA: String of C code
mod = SourceModule("__global__ void add(...) { ... }")
```

While powerful, debugging C strings in Python can be cumbersome.

**Numba** changes this completely:

- You write your GPU kernels in **pure, idiomatic Python syntax!**
  - At runtime, Numba's LLVM-based JIT compiler inspects the Python AST and compiles it directly into NVIDIA PTX machine code.
  - Automatic type inference, memory management, and debugging support.
- 

### The `@cuda.jit` Decorator

Decorating a Python function with `@cuda.jit` marks it for GPU compilation:

```
from numba import cuda

@cuda.jit
def vector_add(a, b, c):
    # Absolute 1D thread position across all blocks:
    idx = cuda.grid(1)
    if idx < c.size:
        c[idx] = a[idx] + b[idx]
```

## Launching the Kernel

Numba uses a specialized syntax to define execution configuration `[blocks, threads]`:

```
import numpy as np

N = 1_000_000
a = np.ones(N, dtype=np.float32)
b = np.ones(N, dtype=np.float32) * 2.0
c = np.empty_like(a)

# Transfer data to GPU memory:
d_a = cuda.to_device(a)
d_b = cuda.to_device(b)
d_c = cuda.device_array_like(a)

# Calculate grid geometry:
threads_per_block = 256
blocks_per_grid = (N + (threads_per_block - 1)) // threads_per_block

# Launch directly!
vector_add[blocks_per_grid, threads_per_block](d_a, d_b, d_c)

# Copy result back:
c = d_c.copy_to_host()
```

---

## Thread Indexing with `cuda.grid()`

Instead of manually typing `blockIdx.x * blockDim.x + threadIdx.x`:

- `cuda.grid(1)`: Returns the absolute 1D thread index.
- `cuda.grid(2)`: Returns `(x, y)` coordinates for 2D matrix processing.

```
@cuda.jit
def matrix_add_2d(A, B, C):
    row, col = cuda.grid(2)
```

```
if row < C.shape[0] and col < C.shape[1]:
    C[row, col] = A[row, col] + B[row, col]
```

---

## Shared Memory Optimization & Thread Synchronization

Global VRAM is located off-chip and requires hundreds of clock cycles to access. **Shared Memory** is an on-chip, ultra-fast memory bank shared by all threads inside a block:

```
@cuda.jit
def fast_stencil(input_data, output_data):
    # Allocate shared on-chip array (SRAM):
    shared_tile = cuda.shared.array(shape=(256,), dtype=np.float32)

    t_id = cuda.threadIdx.x
    g_id = cuda.grid(1)

    # 1. Load data from slow global VRAM into fast on-chip shared cache:
    shared_tile[t_id] = input_data[g_id]

    # 2. Wait until ALL threads in the block have finished loading:
    cuda.syncthreads()

    # 3. Perform calculations using fast shared memory:
    output_data[g_id] = shared_tile[t_id] * 2.0
```

---

## Demo Time

Now, let's see these theoretical topics in action.

- 03\_numba\_cuda\_jit.py —
  1. `demo_numba_cuda_kernel()` — Pure Python GPU kernel with `@cuda.jit` and `cuda.grid(1)`
  2. `demo_numba_shared_memory()` — Fast on-chip shared memory (`cuda.shared.array`) and `cuda.syncthreads()`
  3. `demo_numba_cpu_parallel()` — High-performance multi-core CPU fallback with `@jit(parallel=True)`

---

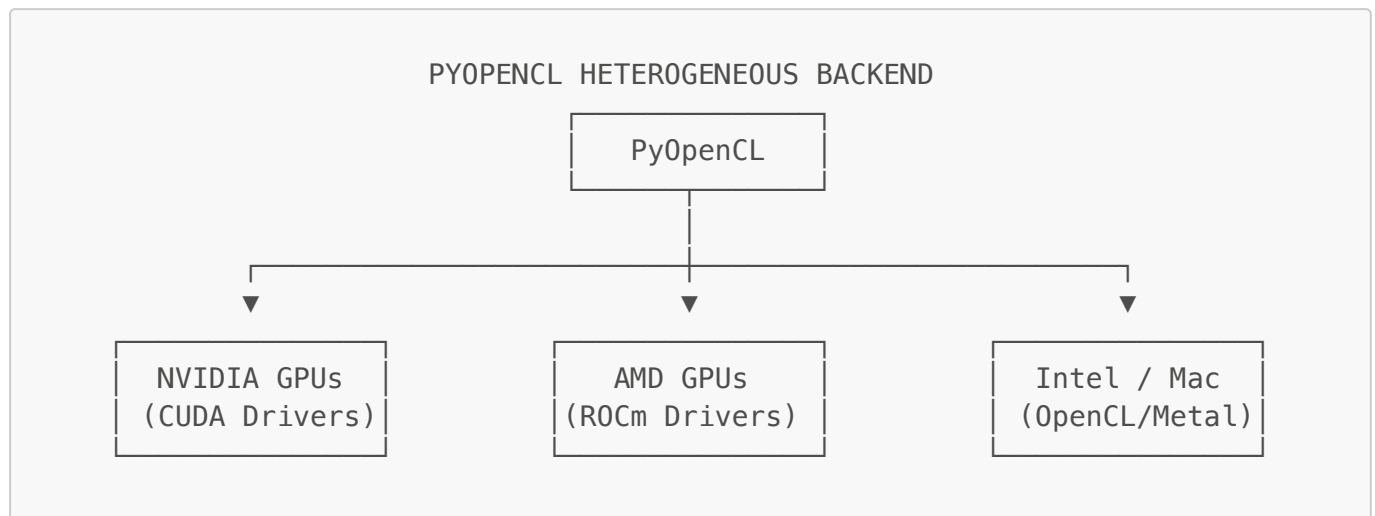
## 2. Cross-Platform Heterogeneous Computing with PyOpenCL

### Why OpenCL? The Vendor Independence Advantage

While CUDA is exceptionally mature, it is locked to **NVIDIA hardware**.

**OpenCL** (Open Computing Language) is an open, royalty-free standard maintained by the **Khronos Group**:

- Runs on **NVIDIA GPUs, AMD Radeon GPUs, Intel GPUs, Apple Silicon GPUs, and x86/ARM CPUs.**
- Ensures your high-performance code is not locked to a single hardware vendor.



## The OpenCL Hardware & Platform Model

OpenCL organizes compute resources as follows:

```
Platform (e.g. "Apple OpenCL" or "NVIDIA CUDA")
├── Device (e.g. "Apple M3 GPU" or "RTX 4090")
│   ├── Context (Resource container: memory, programs, queues)
│   │   ├── CommandQueue (Issues commands to the device)
│   │   └── Buffer (Device memory chunks)
```

| OpenCL Concept       | CUDA Equivalent | Description                                          |
|----------------------|-----------------|------------------------------------------------------|
| <b>Platform</b>      | Driver API      | Software installation of OpenCL from a vendor        |
| <b>Device</b>        | GPU Device      | Physical accelerator chip                            |
| <b>Context</b>       | CUDA Context    | Environment holding memory and compiled kernels      |
| <b>Command Queue</b> | CUDA Stream     | Stream of execution commands sent to the device      |
| <b>Work-Item</b>     | Thread          | Single execution instance                            |
| <b>Work-Group</b>    | Thread Block    | Collection of work-items that can share local memory |
| <b>ND-Range</b>      | Grid            | N-Dimensional execution space (1D, 2D, or 3D)        |

## Compiling and Running OpenCL Kernels

In PyOpenCL, kernels are written in standard OpenCL C and compiled dynamically at runtime:

```
import pyopencl as cl
import numpy as np
```

```
# 1. Setup Context & Queue:
ctx = cl.create_some_context()
queue = cl.CommandQueue(ctx)

# 2. OpenCL C Kernel:
kernel_code = """
__kernel void vector_square(__global const float *in,
                           __global float *out) {
    int gid = get_global_id(0);
    out[gid] = in[gid] * in[gid];
}
"""
prg = cl.Program(ctx, kernel_code).build()

# 3. Memory Buffers:
mf = cl.mem_flags
in_buf = cl.Buffer(ctx, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=host_data)
out_buf = cl.Buffer(ctx, mf.WRITE_ONLY, host_data.nbytes)

# 4. Enqueue Kernel execution across N work-items:
prg.vector_square(queue, (N,), None, in_buf, out_buf)

# 5. Read back:
cl.enqueue_copy(queue, result_data, out_buf)
```

## Demo Time

Now, let's see these theoretical topics in action.

- `04_pyopencl_basics.py` —
  1. `demo_opencl_device_discovery()` — Querying platforms, devices, and compute units
  2. `demo_opencl_kernel_execution()` — Compiling and dispatching OpenCL C kernels

## 3. CUDA vs OpenCL vs CPU Concurrency: Final Comparison

| Criterion            | CPU Multiprocessing / MPI                       | PyCUDA / Numba CUDA                       | PyOpenCL                  |
|----------------------|-------------------------------------------------|-------------------------------------------|---------------------------|
| Hardware             | Multi-core CPUs, Clusters                       | NVIDIA GPUs only                          | NVIDIA, AMD, Intel, Apple |
| Programming Language | Pure Python<br>( <code>multiprocessing</code> ) | Python ( <code>@cuda.jit</code> )<br>or C | OpenCL C                  |
| Concurrency Scale    | 4 - 128 cores per node                          | 1,000 - 10,000+ cores                     | 1,000 - 10,000+ cores     |

| Criterion          | CPU Multiprocessing / MPI | PyCUDA / Numba CUDA             | PyOpenCL                        |
|--------------------|---------------------------|---------------------------------|---------------------------------|
| Vendor Lock-in     | None (Runs everywhere)    | High (NVIDIA hardware)          | None (Standardized)             |
| Ecosystem Maturity | Standard Python           | Maximum (AI/Deep Learning, HPC) | High (Embedded, Cross-platform) |

## 4. 5-Day Course Summary & Grand Wrap-Up

Congratulations on completing the **5-Day Python with Parallel Programming** masterclass! Here is how all pieces fit together:

### THE PARALLEL PYTHON STACK

|                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------|
| Day 5: GPU Acceleration (PyCUDA, Numba CUDA, PyOpenCL)<br>-> Massively parallel data processing (10,000+ cores)                           |
| Day 4: Distributed Python (Celery, SCOOB, Pyro4, RPyC)<br>-> Microservices, distributed queues, cloud cluster tasks                       |
| Day 3: Distributed Clusters & Async (MPI / mpi4py, asyncio, futures)<br>-> Supercomputing message passing & high-concurrency I/O loops    |
| Day 2: Advanced Concurrency (Locks, Conditions, Semaphores, IPC, Pool)<br>-> Safe shared-memory synchronization and multi-process control |
| Day 1: Foundations (Serialization, GIL, Performance Metrics, Amdahl)<br>-> Understanding bottlenecks, memory isolation, and speedup       |

You now possess the theoretical foundations and hands-on coding skills to design, debug, and scale high-performance Python applications across CPUs, GPUs, networks, and supercomputers!

**End of Day 5 — Course Complete!**